HTTP, REST & FastAPI Fundamentals

> Part of the Python API Backend curriculum. This document covers the underlying HTTP/REST concepts, then FastAPI's routing, validation, and dependency model.

1. What a backend API is for

A backend API's job, in this kind of project, is narrow: read the results a data pipeline already computed and persisted, and make them available to other software over a network, in a standard, structured format. It is not where analysis or model training happens — by the time a request reaches the API, the heavy computation is already done; the API's own runtime work is typically just filtering, aggregating, or reshaping already-computed data to answer a specific question.

Building a dedicated API layer (rather than letting a frontend read raw data files directly) buys you a stable, versioned contract that's independent of how the underlying data is stored, the ability to serve more than one client, and a place to put validation, caching, and error-handling logic once instead of duplicating it in every consumer.

2. HTTP fundamentals

HTTP (Hypertext Transfer Protocol) is a request/response protocol: a client sends a request to a URL, and a server sends back a response.

2.1 Methods

Method	Purpose	Idempotent?
`GET`	Retrieve a resource	Yes
`POST`	Create a new resource, or trigger a non-idempotent action	No
`PUT`	Replace a resource entirely	Yes
`PATCH`	Partially update a resource	No (in general)
`DELETE`	Remove a resource	Yes

A data-serving analytics API like the one this curriculum focuses on is almost entirely `GET` — every endpoint answers a read-only question about already-computed data, with no side effects. See the architecture overview's idempotency principle for why the idempotent/non-idempotent distinction matters beyond just categorization: clients and proxies are allowed to automatically retry idempotent requests, but never non-idempotent ones, without explicit confirmation.

2.2 URL structure

- **Path parameters** identify a specific resource, embedded directly in the URL: `/items/{item_id}`.
- **Query parameters** modify or filter a request without being part of the resource identity: `/items?category=X&sort=price`.

2.3 Status codes

Status codes communicate the outcome of a request beyond just the body content — the ones that matter most for a data-serving API:

Code	Meaning	When to use it
`200`	OK	the request succeeded, body contains the result
`400`	Bad Request	the client sent a combination of parameters that doesn't make sense (e.g., a required parameter is missing given some other parameter's value)
`404`	Not Found	the requested *resource* doesn't exist (e.g., an identifier that isn't valid)
`422`	Unprocessable Entity	the request *shape* is invalid (wrong type, value outside an allowed set) — most Python frameworks with typed request validation generate this automatically
`503`	Service Unavailable	the endpoint itself is valid, but the data it needs isn't ready yet — a distinct case from 404, worth its own status code

Getting the status-code semantics right matters more than it might seem — a client (or a developer debugging a client) reads the status code first, before ever looking at the body, to decide how to react. Confusing "the resource doesn't exist" (404) with "the resource exists but its data isn't ready yet" (503) means a client can't distinguish "this will never work" from "try again shortly."

3. REST conventions

REST (Representational State Transfer) is a set of conventions for designing HTTP APIs around **resources** — nouns, not verbs. A resource is addressed by a URL, and the HTTP method determines the action taken on it:

```
GET  /customers/abc123/orders     ✓ a resource ("abc123's orders"), addressed by URL
GET  /getCustomerOrders?id=abc123 ✗ verb-in-the-URL, not RESTful convention
```

Being consistent about this convention across an API's endpoints makes it predictable to a new consumer without needing to read documentation for every single route — once they understand the pattern for one resource, the pattern for another is the same shape.

4. FastAPI fundamentals

FastAPI is a Python web framework built around type hints: you describe request and response shapes with ordinary Python type annotations, and the framework handles validation, JSON serialization, and documentation generation from those annotations automatically.

4.1 Routing

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def get_item(item_id: str):
    ...

@app.get("/items")
def list_items(category: str | None = None, limit: int = 10):
    # category/limit are query parameters — declared as ordinary function arguments
    # with defaults, distinguished from path parameters by not appearing in the path string
    ...
```

Route-matching order matters. Routes are matched in the order they're declared (in FastAPI/Starlette, as in most similar frameworks). If you have both a fixed path (`/items/summary`) and a parameterized one (`/items/{item_id}`) under the same prefix, declare the fixed one **first** — otherwise a request for `/items/summary` will match the parameterized route instead, with `item_id="summary"`, and never reach the handler you actually meant to serve it. This is one of the most common real routing bugs in a framework of this style, and it's invisible until you specifically test for it (see Testing, §6).

4.2 Request/response validation with Pydantic

FastAPI uses **Pydantic** models to define the *shape* of data flowing in and out:

```python
from pydantic import BaseModel

class ItemResponse(BaseModel):
    id: str
    name: str
    value: float | None   # Optional — None is a valid value, becomes JSON null

@app.get("/items/{item_id}", response_model=ItemResponse)
def get_item(item_id: str) -> ItemResponse:
    ...
    return ItemResponse(id=item_id, name="...", value=None)
```

Declaring `response_model` does two things: it validates that whatever the function returns actually matches the declared shape (catching a mismatch at development time rather than shipping malformed JSON to a client), and it feeds FastAPI's automatic OpenAPI/documentation generation (`/docs`, `/openapi.json`) — anyone consuming this API can read exactly what shape to expect without reading the implementation.

A value that's legitimately sometimes absent (e.g., computed from an upstream value that can itself be missing) should be typed `Optional[...]` (or the `X | None` shorthand) — this is what lets you return `None` and have it serialize as JSON `null`, which a typed frontend client can represent explicitly rather than needing a sentinel value like `-1` or `""`.

Restricting a query parameter to a fixed set of allowed values is done with `Literal`:

```
from typing import Literal

Sort = Literal["price", "name", "date"]

@app.get("/items")
def list_items(sort: Sort = "name"):
    ...
```

Any value outside the declared set automatically produces a `422` response — no manual validation code needed.

4.3 Nested models and field validation

Pydantic models can nest, and individual fields can carry their own validation constraints:

```
from pydantic import BaseModel, Field, field_validator

class Address(BaseModel):
    city: str
    postal_code: str = Field(pattern=r"^\d{5}$")   # constrained via a regex pattern

class Customer(BaseModel):
    name: str
    age: int = Field(ge=0, le=120)   # ge/le: greater-than-or-equal / less-than-or-equal bounds
    address: Address                  # nested model — validated recursively

    @field_validator("name")
    @classmethod
    def name_must_not_be_blank(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("name must not be blank")
        return v
```

`Field(...)` attaches constraints (numeric bounds, string patterns, length limits) directly to a field's type annotation; `@field_validator` lets you write arbitrary custom validation logic for a field, run automatically whenever the model is constructed or parsed.

4.4 Dependency injection

FastAPI has a built-in **dependency injection** system (`Depends`) for logic that many route handlers need to share — reading a query parameter with shared defaults, checking an API key, opening and closing a database session:

```
from fastapi import Depends

def get_pagination(page: int = 1, page_size: int = 20) -> dict:
    return {"page": page, "page_size": page_size}

@app.get("/items")
def list_items(pagination: dict = Depends(get_pagination)):
    ...
```

`Depends(get_pagination)` tells FastAPI to call `get_pagination` (itself a function that can declare its own query parameters, exactly like a route handler) and inject its return value into `list_items`. This is FastAPI's mechanism for avoiding repeated boilerplate across many endpoints that need the same shared setup — write the shared logic once, as a plain function, and reference it via `Depends` wherever it's needed.

4.5 Async vs. sync route handlers

```
@app.get("/items")
async def list_items_async():
    result = await some_async_database_call()
    return result

@app.get("/other")
def list_items_sync():
    result = some_regular_blocking_call()
    return result
```

FastAPI supports both `async def` and plain `def` route handlers. Use `async def` when the handler awaits genuinely asynchronous I/O (an async database driver, an async HTTP client call to another service) — this lets the server handle other requests while waiting on I/O, rather than blocking a whole worker on it. If your handler only does synchronous work (as is typical for a service that just reads an already-loaded, in-memory/cached DataFrame and filters it), a plain `def` handler is simpler and FastAPI runs it in a way that still doesn't block the rest of the event loop. **Never call a blocking, synchronous operation directly inside an `async def` handler** (e.g., a synchronous file read with no async equivalent) — it will block the entire event loop, defeating the purpose of using `async` at all; either keep the handler as plain `def` (FastAPI handles it appropriately) or use an async-native equivalent.

4.6 CORS

CORS (Cross-Origin Resource Sharing): browsers block a web page from calling an API on a different origin (domain/port) unless that API explicitly allows it. If your frontend and API are served from different origins (very

common in local development — e.g., a frontend dev server on one port, an API on another), you need to configure this on the API side:

```
from fastapi.middleware.cors import CORSMiddleware


app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],   # be explicit; avoid "*" once real user data is involved
    allow_methods=["GET"],
    allow_headers=["*"],
)
```

4.7 Custom middleware

Custom middleware is the general mechanism for logic that needs to run on every request before routing happens:

```
class TimingMiddleware:
    def __init__(self, app):
        self.app = app

    async def __call__(self, scope, receive, send):
        if scope["type"] != "http":
            return await self.app(scope, receive, send)
        import time
        start = time.time()
        await self.app(scope, receive, send)
        print(f"Request took {time.time() - start:.3f}s")

app.add_middleware(TimingMiddleware)
```

A common real use case is a service that needs to be reachable both directly and behind a reverse proxy that forwards requests under some URL prefix — middleware is where you'd strip that prefix before the router sees the request path. Writing this kind of middleware requires being careful about *exact* path-boundary matching: a prefix of `/app` should match `/app/x` but must not match `/application/x` just because it happens to start with the same characters — a plain `path.startswith(prefix)` is not sufficient on its own; check that what follows the prefix is either nothing or a `/`.

5. Running the API

FastAPI apps are run with an **ASGI server** — `uvicorn` is the common choice. ASGI (Asynchronous Server Gateway Interface) is the modern successor to WSGI, the older standard most Python web-framework-to-server

interfaces used; the practical difference that matters day-to-day is that ASGI natively supports `async` / `await` and things like WebSockets, not just simple request/response.

```
uvicorn app.main:app --reload --port 8000
```

`--reload` restarts the server automatically when source files change — useful in development, never in production (see Deployment, §2 for the production equivalent).

6. Core libraries summary

Library	Used for
FastAPI	the web framework — routing, request/response handling, dependency injection, automatic docs
Pydantic	request/response validation and serialization
uvicorn	the ASGI server that actually runs the app in development

See also

- Python API Backend index
- API Design & Best Practices — data loading, caching, error handling, and API design conventions built on top of these fundamentals